

Code Explanation — collect_data.py

Date: 2026-08-26 **File:** `training/collect_data.py` **Purpose:** Webcam-based data collection tool. Records hand-landmark sequences for the 15-word EthSL lexicon using MediaPipe's Tasks API (`HandLandmarker`), saving one JSON file per repetition.

Module docstring

States why the Tasks API was chosen over the legacy `mediapipe.solutions` API (model-version traceability for the published dataset), the v1 scope decision (hands-only, no pose/body landmarks), and the keyboard controls (`SPACE / r / n / p / q`).

Imports

Standard library: `[json, sys, time, dataclass, datetime, Path, Third-party: cv2 (webcam), mediapipe + BaseOptions/vision from mediapipe.tasks.python (the modern Tasks API namespace — not mediapipe.solutions, which this MediaPipe build doesn't even include). Local: [CLASSES, REPS_PER_CLASS, VALID_SIGNER_IDS from class_list.py.`

Constants

`[ROOT_DIR / DATA_RAW_DIR / MODEL_PATH]` — paths computed relative to the file's own location, so the script works regardless of the working directory it's launched from.
`[MODEL_VERSION]` — stamped into every saved rep for traceability.
`[RECORD_SECONDS, COUNTDOWN_SECONDS, TARGET_FPS, FRAME_INTERVAL_MS]` — timing values from the data collection protocol.
`[MIN_DETECTION_RATIO_WARN]` — the 50% threshold below which a rep gets flagged as possibly bad.

`[CAM_INDEX, FRAME_W, FRAME_H]` — webcam configuration.
`[HAND_CONNECTIONS]` — the 21-point skeleton's line topology, sourced from `mediapipe.tasks.python.vision.HandLandmarksConnections` (not the legacy `solutions.hands` module, which isn't present in this install) — used only for drawing the on-screen overlay.

SessionState (dataclass)

Holds the three things that change during a session: `[signer_id, class_idx]` (which of the 15 words is currently selected), `[last_saved_path]` (for the redo feature).

MonotonicClock

MediaPipe's VIDEO running mode requires every frame's timestamp to strictly increase. This class hands out one shared, ever-increasing millisecond counter used by every call to `[detect()]` across the whole session (idle preview, countdown, recording) — prevents timestamp ordering errors.

build_landmarker()

Loads `[hand_landmarker.task]`, exits with a clear message if it's missing, configures `[HandLandmarkerOptions]` (VIDEO mode, 2 hands max, 0.5 confidence thresholds), returns the ready detector.

get_signer_id()

Prompts until a valid ID (`[s1-s5]`) is entered.

existing_rep_count()

Counts existing `[.json]` files for a signer/class on disk — this is what makes the script resumable across sessions with no separate progress file needed.

detect()

Converts a BGR OpenCV frame to MediaPipe's expected RGB `Image` format and calls `detect_for_video()`. Every other function funnels through this one so the conversion logic exists in exactly one place.

`result_to_hand_slots()`

Converts MediaPipe's raw result into the `hand_1/hand_2` dict structure that gets saved. **Deliberately does not assume `hand_1` is always the same physical hand** — MediaPipe's slot order isn't guaranteed stable frame to frame. The real hand identity lives in the `handedness` field inside each entry; sorting that out is `preprocess.py`'s job, not this file's — raw collection stays as close to "exactly what MediaPipe returned" as possible.

`draw_overlay()`

Purely visual: draws the skeleton (landmark dots + connecting lines) on the displayed frame. Has no effect on saved data.

`draw_hud()`

Purely visual: draws the top status bar (current word, hand-count note, class/rep progress) and bottom control hints. Same function renders every status state (ready / countdown / recording / done) via its `status_text/status_color` arguments.

`save_rep()`

Assembles the final JSON record and writes it to

`data/raw/{signer_id}/{class_label}/{signer_id}_{class_label}_{rep_num:03d}.json`. Returns the path so `main()` can support redo.

`run_countdown()`

The "get ready" phase. Shows the live feed with a countdown message for `COUNTDOWN_SECONDS`. Does **not** call `detect()` and does **not** save anything —

exists purely to prepare the signer, not to capture data.

`record_window()`

The actual capture loop. For `RECORD_SECONDS`, reads frames, detects, converts via `result_to_hand_slots()`, appends to a list, shows the live overlay + "RECORDING..." status. Returns the completed frame list for `save_rep()`.

`main()`

Orchestrates everything: gets signer ID, opens webcam, builds the landmarker, then loops showing the idle preview and handling keypresses — `q/esc` → exit `n/p` → change selected class `r` → delete last saved rep `SPACE` (if current class isn't already at 18/18) → countdown → record → save → print detection-ratio feedback, warning if low

The `finally` block guarantees the camera, display window, and detector are released even on a crash — prevents a hung webcam process.

Bottom

`if __name__ == "__main__":` — standard entry point guard.

Known issues encountered / resolved

2026-08-26: `ImportError: cannot import name 'CLASSES' from 'class_list'` on first run — caused by `class_list.py` existing as an empty file (never populated), not by any issue with `hand_landmarker.task` or the `HandLandmarker` API. Resolved by populating `class_list.py` with the locked 15-word list.